



ASSIGNMENT 2.1

Del directorio vacio al proyecto reproducible

Un proyecto pequeno para practicar entorno, dependencias y trazabilidad local

DURACION	MODALIDAD	PREREQUISITOS
45-50 min	Parejas, terminal Bash	uv y Git disponibles; no necesitas trabajo previo

1. Encargo

Construir env-demo desde un directorio vacio y demostrar que una sola base de codigo se comporta de forma predecible en dev, pre y pro.

Regla de continuidad

Trabaja solo en local: esta practica no usa remoto ni parte del ejercicio de Wine.

2. Punto de partida

- La pareja comparte pantalla y alterna quien escribe los comandos y quien contrasta la evidencia.
- El resultado debe usar un unico codigo, un unico uv.lock y un unico .venv para los tres entornos.

3. Material que recibes

- Una terminal Bash situada en un directorio de trabajo vacio
- uv y Git ya instalados
- Los dos cuadros de codigo incluidos en este documento

4. Tareas

01 Crea el proyecto - Crea env-demo, entra en el directorio, ejecuta git init y despues uv init --package --vcs none .

02 Declara dependencias - Crea .gitignore con .venv/, __pycache__/, .pytest_cache/ y .ruff_cache/. Añade Rich, Pytest y Ruff con los comandos indicados.

03 Revisa la estructura - Localiza pyproject.toml, uv.lock, src/env_demo y tests. No crees carpetas distintas por entorno ni repitas dependencias.

04 Implementa el comando - Sustituye src/env_demo/main.py por este codigo. La variable APP_ENV acepta dev, pre o pro, normaliza la entrada y Rich muestra el mensaje.

src/env_demo/main.py

```
import os

from rich import print

VALID_ENVIRONMENTS = {"dev", "pre", "pro"}

def environment_message(environment: str) -> str:
    normalized = environment.strip().lower()
    if normalized not in VALID_ENVIRONMENTS:
        raise ValueError("APP_ENV debe ser dev, pre o pro")
    return f"Entorno activo: {normalized.upper()}"

def main() -> None:
    print(environment_message(os.getenv("APP_ENV", "dev")))

if __name__ == "__main__":
    main()
```

05 Prueba el contrato - Crea tests/test_main.py con estos casos. El parametrizado cubre los tres valores admitidos y el ultimo protege el error.

```
tests/test_main.py

import pytest

from env_demo.main import environment_message

@pytest.mark.parametrize("environment", ["dev", "pre", "pro"])
def test_known_environment(environment: str) -> None:
    assert environment.upper() in environment_message(environment)

def test_unknown_environment() -> None:
    with pytest.raises(ValueError, match="APP_ENV"):
        environment_message("local")
```

06 Documenta y registra - Escribe en README como instalar, ejecutar y comprobar. Haz tres commits locales, en orden, con las intenciones exactas indicadas abajo.

5. Entrega

Evidencia	Que debe permitir comprobar
Proyecto local	pyproject.toml, uv.lock, paquete src, tests, README y .gitignore.
Historial	Tres commits locales pequenos y legibles, sin configurar remoto.
Comprobacion	Salida de los tres entornos, Pytest, Ruff y controles de Git.

6. Criterios de aceptacion

- [] Los tres entornos ejecutan exactamente el mismo paquete y cambian solo mediante APP_ENV.
- [] La entrada local provoca ValueError y los cuatro casos de prueba pasan.
- [] Rich es dependencia normal; Pytest y Ruff son dependencias de desarrollo.
- [] .venv no aparece entre los archivos versionados y uv.lock si aparece.
- [] README permite repetir el trabajo desde un checkout limpio.

7. Rubrica orientativa

Criterio	Puntos
Proyecto y dependencias	3
Comando y entornos	3
Tests y calidad	2
README e historial	2

8. Comprobacion

```
mkdir env-demo && cd env-demo
git init
uv init --package --vcs none .
printf '.venv/\n__pycache__/\n.pytest_cache/\n.ruff_cache/\n' > .gitignore

uv add rich
uv add --dev pytest ruff

APP_ENV=dev uv run python -m env_demo.main
APP_ENV=pre uv run python -m env_demo.main
APP_ENV=pro uv run python -m env_demo.main

uv run pytest
uv run ruff check .

git log --oneline
git ls-files .venv
```

9. Notas y limites

- Commits, en orden: bootstrap uv project; add environment-aware command; add tests and usage documentation.
- Una sola base de codigo, uv.lock y .venv; no prepares variantes dev/pre/pro.
- Termina sin remoto: no publiques ni abras una revision.

Referencia en el repositorio

Assignment autocontenido; configuracion en pyproject.toml